Consider a binary classification problem with inputs $\boldsymbol{x} \in \mathbb{R}^d$ and output labels $y \in \{0, 1\}$. A dataset is called **linearly separable** if there exist weights $\boldsymbol{w} \in \mathbb{R}^d$ and bias $w_0 \in \mathbb{R}$ s.t. every input is correctly classified by the binary threshold decision rule:

$$\hat{y} = \begin{cases} 1, & \text{if } \boldsymbol{w} \cdot \boldsymbol{x} + w_0 > 0 \\ 0, & \text{if } \boldsymbol{w} \cdot \boldsymbol{x} + w_0 \leq 0 \end{cases}$$

In other words, a dataset with binary labels is linearly separable if a single hyperplane (a straight line when $d = 2$) can separate all positive examples from all negative examples.

In this lecture, we show that AND, OR, and NAND, for example, are all linearly separable functions, and as such, can be represented by a simple perceptron. On the other hand, XOR is not linearly separable. But fear not: XOR can be represented by a multilayer perceptron.

**The Perceptron** The binary threshold decision rule is an instance of the Heaviside step function, namely: for $z \in \mathbb{R}$,

$$H(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z \leq 0 \end{cases}$$

A simple perceptron uses $H$ as its output activation, and thus computes

$$\hat{y} = H(\boldsymbol{w} \cdot \boldsymbol{x} + w_0)$$

**The AND Function** The AND function returns 1 only when both inputs are 1.

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

A perceptron that represents the AND function requires bias $w_0$ and weights $w_1$ and $w_2$ s.t.

$$y = H(w_0 + w_1 x_1 + w_2 x_2)$$

Choose

$$w_0 = -1.5 \qquad w_1 = 1 \qquad w_2 = 1$$

The equation of this perceptron is

$$y = H(x_1 + x_2 - 1.5)$$

And the decision boundary, which is indeed linear, is

$$x_1 + x_2 = 1.5$$

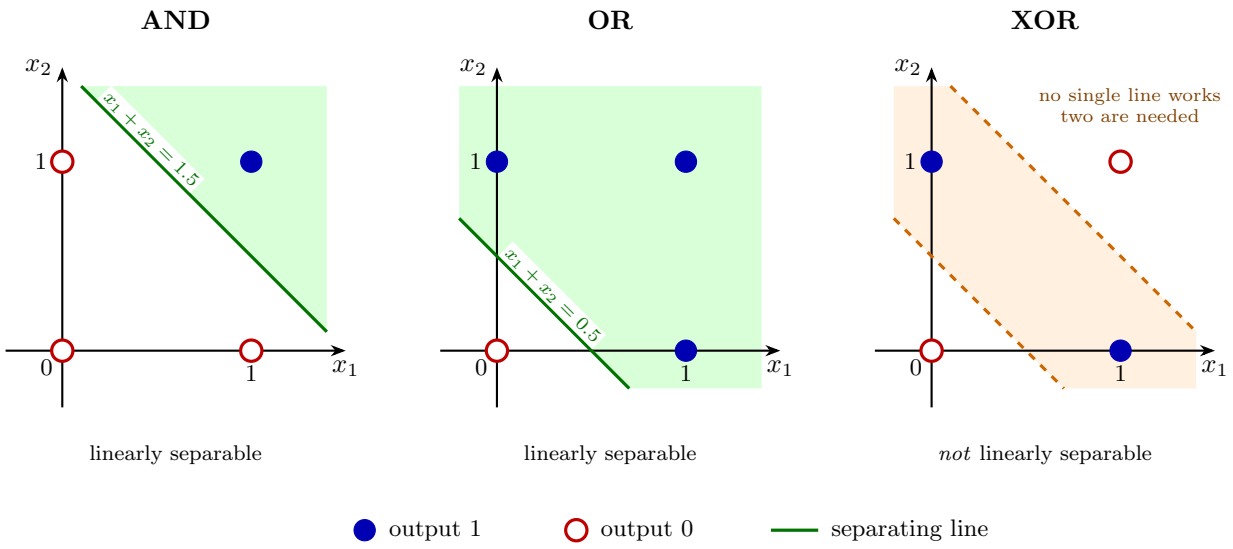



Figure 1: The AND and OR functions are linearly separable, while XOR is not.

Let's verify that this perceptron / linear decision boundary linearly separates the AND data:

$$\begin{aligned}(0,0): &\quad H(-1.5) = 0\\ (0,1): &\quad H(-0.5) = 0\\ (1,0): &\quad H(-0.5) = 0\\ (1,1): &\quad H(0.5) = 1\end{aligned}$$

The point $(1,1)$ lies on the positive side of this boundary, while all three other points lie on the negative side. Hence, all inputs are classified correctly, which implies that AND is linearly separable.

**The OR Function** The OR function returns 1 whenever at least one input is 1.

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

As above, a perceptron that represents the OR function requires bias $w_0$ and weights $w_1$ and $w_2$ s.t.

$$y = H(w_0 + w_1 x_1 + w_2 x_2)$$

Choose

$$w_0 = -0.5 \qquad w_1 = 1 \qquad w_2 = 1$$

The equation of this perceptron is

$$y = H(x_1 + x_2 - 0.5)$$

And the decision boundary, which is indeed linear, is

$$x_1 + x_2 = 0.5.$$

Let's verify that this perceptron / linear decision boundary linearly separates the OR data:

$$\begin{aligned}(0,0): &\quad H(-0.5) = 0\\ (0,1): &\quad H(0.5) = 1\\ (1,0): &\quad H(0.5) = 1\\ (1,1): &\quad H(1.5) = 1\end{aligned}$$

The point $(0,0)$ lies on the negative side of this boundary, while all three other points lie on the positive side. Hence, all inputs are classified correctly, which implies that OR is linearly separable.

**The XOR Function** The XOR (exclusive OR) function returns 1 when its inputs differ and 0 when they are the same.

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Pictorially (see Figure 1), XOR assigns opposite labels to diagonally opposite corners of the unit square:

$$\begin{aligned}y = 0: &\quad (0,0), (1,1),\\ y = 1: &\quad (0,1), (1,0).\end{aligned}$$

**Theorem** No simple perceptron can represent XOR.

**Proof** Suppose, for the sake of contradiction, that XOR could in fact be represented by

$$y = H(w_0 + w_1 x_1 + w_2 x_2)$$

for some choice of $w_0, w_1, w_2 \in \mathbb{R}$.

On the input $(0,0)$, the output must be 0, which implies

$$w_0 \leq 0 \tag{1}$$

On the input $(1,1)$, the output again must be 0, which implies

$$w_0 + w_1 + w_2 \leq 0 \tag{2}$$

On the input $(1,0)$, the output must be 1, which implies

$$w_0 + w_1 > 0 \tag{3}$$

On the input $(0,1)$, the output again must be 1, which implies

$$w_0 + w_2 > 0 \tag{4}$$

Summing inequalities (3) and (4) yields

$$2w_0 + w_1 + w_2 > 0$$

Equivalently,

$$w_0 + w_1 + w_2 > -w_0$$

But by inequality (1), $w_0 \leq 0$. Equivalently, $-w_0 \geq 0$. Therefore,

$$w_0 + w_1 + w_2 > 0 \tag{5}$$

But inequalities (2) and (5) contradict one another. Hence, no weights $w_0, w_1, w_2 \in \mathbb{R}$, i.e., no single linear decision boundary, can correctly classify all four XOR inputs. In other words, XOR is not linearly separable.

**Representing XOR with a Neural Network** Although a simple perceptron cannot represent XOR, a neural network with two layers can. The key insight is to transform the original inputs into an intermediate representation that a simple perceptron can classify: i.e., one that is linearly separable.

The XOR function can be expressed in terms of OR, AND, and NAND.

| $x_1$ | $x_2$ | $x_1 \vee x_2$ | $\neg(x_1 \wedge x_2)$ | $(x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 |

In other words, $\text{XOR}(x_1, x_2) = \text{OR}(x_1, x_2) \wedge \text{NAND}(x_1, x_2)$.

Recall that a simple perceptron can represent both OR and AND, respectively, as follows:

$$h_{\text{OR}} = H(x_1 + x_2 - 0.5)$$

$$h_{\text{AND}} = H(x_1 + x_2 - 1.5)$$

Similarly, a simple perceptron can represent NAND as follows:

$$h_{\text{NAND}} = H(-x_1 - x_2 + 1.5)$$

In other words, NAND is linearly separable. Thus, by applying AND to the output of OR and NAND, we can build an XOR classifier using simple perceptrons.

The network in Figure 2 illustrates this construction. It comprises:

- Two input neurons, $x_1$ and $x_2$
- Two hidden neurons, $h_{\text{OR}}(x_1, x_2)$ and $h_{\text{NAND}}(x_1, x_2)$
- One output neuron, $y = h_{\text{AND}}(h_{\text{OR}}, h_{\text{NAND}})$

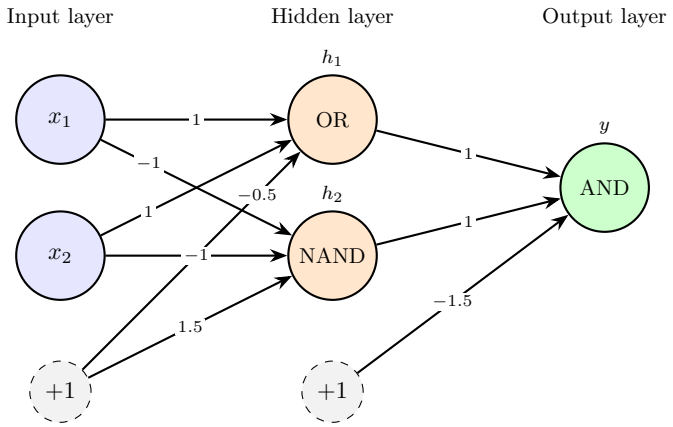



Figure 2: XOR as a two-layer perceptron

Each of the two neurons $h_1$ and $h_2$ in the hidden layer transforms the input data. The function $h_1$ is the OR transformation, while the function $h_2$ is the NAND transformation. The ensuing hidden representation is

| $(x_1, x_2)$ | $(h_1, h_2)$ |
|---|---|
| $(0, 0)$ | $(0, 1)$ |
| $(0, 1)$ | $(1, 1)$ |
| $(1, 0)$ | $(1, 1)$ |
| $(1, 1)$ | $(1, 0)$ |

In this representation, the two positive XOR examples map to $(1, 1)$, while the two negative XOR examples map to $(0, 1)$ and $(1, 0)$. The data are now linearly separable using AND's linear decision boundary, namely

$$h_1 + h_2 = 1.5$$

Therefore, this network combines multiple linear decision boundaries to implement XOR, a function that is not linearly separable in the original input space.

In summary,

- The AND and OR functions are linearly separable, and as such, a simple perceptron can classify their inputs correctly using a single linear decision boundary.
- XOR is not linearly separable, because its positive and negative examples occupy opposite corners of the input square, so that a single linear separation is impossible.
- Although simple perceptrons can only represent linearly separable functions, a multilayer perceptron with one hidden layer can represent XOR, a nonlinearly separable function.

The takeaway from this discussion is: A multilayer perceptron (i.e., one with at least one hidden layer) can classify nonlinearly separable data, even when all local decision boundaries remain linear.